

Cheat Sheet UAS — MediScan AI (Klasifikasi Kanker Payudara MobileNetV2)

1. Fondasi Teori Khusus UAS

Relasi AI, ML, DL (Analoginya seperti Payung/Himpunan)

- **AI** adalah payung terbesar: segala teknik yang membuat mesin bisa meniru kecerdasan manusia.
- **Machine Learning** adalah sub-himpunan AI: mesin belajar pola dari data tanpa diprogram eksplisit.
- **Deep Learning** adalah sub-himpunan ML: menggunakan jaringan saraf bertingkat (banyak lapisan) untuk memproses data kompleks seperti citra.

Contoh konkret: AI adalah tujuannya (mengenal tumor). ML adalah pendekatannya (belajar dari dataset). DL adalah alatnya (MobileNetV2).

Tabel Perbedaan ML vs DL

Aspek	Machine Learning	Deep Learning
Ekstraksi fitur	Manual — perlu domain expert (misal: texture, shape)	Otomatis — jaringan belajar fitur sendiri dari piksel mentah
Jumlah data	Cukup dengan ratusan hingga ribuan sampel	Butuh ribuan hingga jutaan sampel (atau pakai transfer learning)
Kebutuhan komputasi	CPU sudah cukup, training cepat	GPU diperlukan, training bisa berjam-jam
Contoh algoritma	SVM, Random Forest, Logistic Regression	CNN, MobileNetV2, ResNet

Poin jual DL untuk proyek ini: MobileNetV2 sudah pre-trained di ImageNet (1.4 juta gambar), jadi tidak perlu data raksasa — cukup fine-tune untuk citra USG.

2. Langkah Sistematis Membangun Model MediScan AI

Flowchart Hafalan (Urut, Jangan Dilewati)

1. **Load Dataset ZIP** — Admin upload file ZIP berisi folder per kelas (benign, malignant, normal). Sistem validasi struktur folder, filter file mask/annotation, cek minimal 20 gambar per kelas.
2. **Preprocessing via ImageDataGenerator** — Dua generator terpisah (train + validation):
 - Train: `preprocess_input` + augmentasi (rotation 20°, shift 15%, zoom 15%, shear 15%, brightness, horizontal flip)
 - Validation: hanya `preprocess_input` — tidak ada augmentasi
 - Split data menggunakan parameter `validation_split` (default 30%)

3. Arsitektur Two-Phase MobileNetV2 — Ada 2 fase dengan tujuan berbeda:

Fase 1 — Frozen (setengah total epoch)

- Base model MobileNetV2 di-freeze (`trainable=False`)
- Hanya classifier head yang dilatih: `Dense(256, ReLU) → Dropout(0.5) → Dense(128, ReLU) → Dropout(0.3) → Dense(3, Softmax)`
- Learning rate: 1e-4 (default)
- Tujuan: melatih kepala klasifikasi dari nol tanpa merusak fitur ImageNet

Fase 2 — Fine-Tuning (sisanya epoch)

- Base model di-unfreeze dari layer 120 ke atas
- Learning rate diturunkan 10x (1e-5) agar perubahan bobot tidak drastis
- Tujuan: adaptasi fitur 高层 MobileNetV2 ke domain citra USG

4. SparseFocalLoss — Custom loss function:

- `gamma=2.0` — mengurangi kontribusi sampel yang sudah mudah diklasifikasi, fokus ke sampel sulit
- `alpha=0.25` — bobot untuk kelas minoritas (malignant memiliki 421 file vs benign 891)
- Alasan pakai SparseFocalLoss bukan CrossEntropy biasa: dataset tidak seimbang, kesalahan pada kelas malignant lebih berbahaya secara klinis

5. Class Weights — Bobot tambahan tiap kelas: `total_samples / (n_classes x count_per_class)`. Kelas dengan jumlah sedikit (malignant, normal) mendapat bobot lebih besar.

6. Evaluasi — Setelah training selesai:

- Confusion matrix (PNG + JSON)
- Classification report (precision, recall, f1-score per kelas)
- Metrics: accuracy, precision macro, recall macro, f1 macro
- History chart (accuracy & loss per epoch)

7. Penyimpanan Model — File yang disimpan di `storage/models/`:

- `model_<id>.keras` — file model TensorFlow
- `config_<id>.json` — parameter training (epochs, batch_size, lr, dll)
- `history_<id>.json` — riwayat akurasi/loss per epoch
- `confusion_matrix_<id>.png + .json` — visualisasi + data mentah
- `classification_report_<id>.json` — precision/recall/f1 per kelas

3. Panduan Penjelasan Kode Per Baris (Code Walkthrough)

3a. Pembekuan Layer — Fase 1 (`trainer.py:258-274`)

```
base_model = MobileNetV2(weights="imagenet", include_top=False, pooling="avg",
                          input_shape=(image_size, image_size, 3))
base_model.trainable = False
```

- `weights="imagenet"`: Memuat bobot yang sudah dilatih di 1.4 juta gambar ImageNet. Ini **transfer learning** — kita tidak mulai dari nol.
- `include_top=False`: Membuang layer klasifikasi asli ImageNet (1000 kelas) karena kita hanya punya 3 kelas (benign, malignant, normal).
- `pooling="avg"`: GlobalAveragePooling2D menggantikan Flatten — mengurangi parameter drastis, mencegah overfitting.
- `trainable = False`: Semua layer MobileNetV2 dibekukan. Hanya bobot classifier head (Dense + Dropout) yang berubah di Fase 1.

3b. Pembukaan Layer 120+ — Fase 2 / Fine-Tuning (`trainer.py:286-294`)

```
base_model.trainable = True
for layer in base_model.layers[:120]:
    layer.trainable = False
model.compile(optimizer=Adam(learning_rate=lr * 0.1), ...)
```

- `base_model.trainable = True`: Aktivasi ulang base model secara keseluruhan.
- `layers[:120]` tetap dibekukan: Layer awal (0-120) menangkap fitur dasar seperti tepi, sudut, tekstur — fitur ini universal dan tidak perlu diubah.
- Layer 120+ diaktifkan: Layer akhir MobileNetV2 menangkap fitur spesifik domain (bentuk organ, tekstur jaringan). Layer ini yang perlu diadaptasi ke citra USG.
- `lr * 0.1`: Learning rate diturunkan dari 1e-4 (default) menjadi 1e-5 agar fine-tuning tidak merusak bobot yang sudah matang (prevents catastrophic forgetting).

3c. SparseFocalLoss (`trainer.py:36-58`)

```
class SparseFocalLoss(tf.keras.losses.Loss):
    def __init__(self, gamma=2.0, alpha=0.25, from_logits=False):
```

Mengapa `gamma=2.0`?

- Focal Loss dirancang untuk class imbalance. Gamma mengontrol seberapa cepat loss dari sampel mudah dikurangi.
- `gamma=0` = CrossEntropy biasa. `gamma=2.0` adalah nilai standar empiris — cukup agresif mengurangi kontribusi sampel mudah tanpa mengabaikannya total.

Mengapa `alpha=0.25`?

- Alpha adalah bobot kelas positif (atau kelas fokus). Alpha < 0.5 memberi prioritas pada kelas minoritas.

- Dalam konteks kami: kelas malignant adalah prioritas klinis tertinggi. Kesalahan mendiagnosis malignant sebagai benign (false negative) bisa fatal. $\alpha=0.25$ memastikan model memberikan perhatian lebih ke kelas malignant.

Cara kerjanya dalam kode:

```
focal_weight = tf.pow(1.0 - y_pred, self.gamma) * y_true_one_hot
focal_loss = self.alpha * focal_weight * cross_entropy
```

- Jika model sudah yakin benar (prob mendekati 1), $(1 - y_pred)^{2.0}$ mendekati 0 → loss kecil.
- Jika model salah atau ragu-ragu (prob kecil), $(1 - y_pred)^{2.0}$ besar → loss tetap besar.
- Alpha mengalikan hasil ini untuk memberi bobot prioritas pada kelas tertentu.

3d. Penyimpanan Config, CM, dan Report ([trainer.py:316-358](#))

Config JSON:

```
config_path = os.path.join(MODELS_DIR, f"config_{model_id}.json")
with open(config_path, "w") as f:
    json.dump(cfg, f, indent=2)
```

- `cfg` berisi: `dataset_filename`, `epochs`, `validation_split`, `batch_size`, `image_size`, `learning_rate`.
- Disimpan agar di halaman Admin Models, tombol Info bisa menampilkan parameter training yang digunakan.
- Berguna untuk audit dan reproduksibilitas — dosen bisa tanya parameter apa yang dipakai.

Confusion Matrix JSON:

```
cm_data_path = save_path.replace(".png", ".json")
with open(cm_data_path, "w") as f:
    json.dump({"matrix": cm.tolist(), "class_names": class_names}, f)
```

- Data mentah confusion matrix disimpan sebagai JSON agar frontend bisa menghitung analisis dinamis (akurasi global, sensitivitas per kelas, rekomendasi optimasi).
- PNG hanya untuk visualisasi. JSON untuk kalkulasi.

Classification Report JSON:

```
report = classification_report(y_true, y_pred_classes, target_names=class_names,
                              output_dict=True)
with open(save_path, "w") as f:
    json.dump(report, f, indent=2)
```

- `output_dict=True` mengembalikan dictionary, bukan string — lebih mudah diolah frontend.

- Berisi precision, recall, f1-score, support untuk setiap kelas + macro avg + weighted avg + accuracy.
-

4. Tips Menghadapi Pertanyaan Jebakan Dosen

Q1: "Mengapa memilih MobileNetV2, bukan arsitektur CNN lain seperti VGG16 atau ResNet?"

Draf jawaban: "Untuk tiga alasan. Pertama, MobileNetV2 menggunakan depthwise separable convolution yang mengurangi parameter hingga 10x lipat dibanding VGG16, sehingga lebih ringan dan cepat. Kedua, bottleneck residual blocks mempertahankan akurasi tinggi meski parameter lebih sedikit — akurasi final kami 81.11% membuktikan ini. Ketiga, karena deployment targetnya adalah web (bukan superkomputer), model harus ringan. VGG16 punya 138 juta parameter, MobileNetV2 hanya 3.4 juta — perbedaan 40x lebih ramping tanpa mengorbankan performa secara signifikan."

Q2: "Mengapa menggunakan 2 fase training? Kenapa tidak langsung fine-tuning saja?"

Draf jawaban: "Fase 1 melatih classifier head dari bobot acak (random initialized) dengan base model dibekukan. Jika langsung fine-tuning, gradient dari head yang masih acak akan merusak bobot ImageNet yang sudah matang — ini disebut catastrophic forgetting. Fase 1 memberi kesempatan head belajar pola dasar citra USG terlebih dahulu. Setelah head stabil, baru Fase 2 membuka layer akhir MobileNetV2 dengan learning rate kecil untuk adaptasi halus. Hasilnya: training lebih stabil dan konvergen lebih baik."

Q3: "Mengapa akurasi 81.11% tidak menjadi satu-satunya tolok ukur? Bukankah akurasi adalah yang utama?"

Draf jawaban: "Di domain medis, akurasi bisa menipu karena class imbalance. Dataset kami memiliki 891 gambar benign vs 421 malignant vs 266 normal. Model yang memprediksi semua gambar sebagai 'benign' akan mendapat akurasi sekitar 56% — tetapi secara klinis itu berbahaya karena semua kasus malignant terlewatkan. Karena itu kami menggunakan precision, recall, dan f1-score per kelas, terutama untuk kelas malignant. Prioritas kami: false negative malignant harus seminimal mungkin, meskipun akurasi global sedikit turun. Keselamatan pasien di atas segalanya."

Q4: "Kenapa menggunakan SparseFocalLoss? Apa bedanya dengan CrossEntropy biasa?"

Draf jawaban: "CrossEntropy biasa memberikan bobot yang sama untuk semua sampel. Dalam dataset tidak seimbang, model cenderung fokus ke kelas mayoritas (benign) dan mengabaikan malignant. SparseFocalLoss menambahkan dua mekanisme. Pertama, $\gamma=2.0$ mengurangi loss dari sampel yang sudah mudah ditebak, sehingga model dipaksa fokus ke sampel sulit. Kedua, $\alpha=0.25$ memberi bobot prioritas ke kelas minoritas. Tanpa FocalLoss, model kami hanya mencapai sekitar 70% recall untuk malignant. Dengan FocalLoss, recall malignant naik signifikan."

Q5: "Data augmentation yang digunakan apa saja? Mengapa tidak menggunakan augmentasi yang lebih agresif?"

Draf jawaban: "Kami menggunakan rotation 20 derajat, width/height shift 15%, shear 15%, zoom 15%, brightness range 0.8-1.2, dan horizontal flip. Untuk citra USG payudara, orientasi anatomis tidak memiliki arah baku — tumor bisa muncul di sisi mana pun — jadi horizontal flip aman digunakan. Kami tidak menggunakan augmentasi lebih agresif seperti cutout atau mixup karena citra USG memiliki tekstur medis yang detail;

augmentasi terlalu agresif bisa menghilangkan informasi diagnostik penting. Augmentasi diterapkan sebagai preprocessing di ImageDataGenerator, bukan sebagai layer model, untuk menjaga kompatibilitas."

Ringkasan Angka Penting yang Wajib Dihafal

Parameter	Nilai
Akurasi validasi final	81.11%
Loss validasi final	0.4499
Jumlah epoch total	20 (10 frozen + 10 fine-tune)
Learning rate Fase 1	1e-4
Learning rate Fase 2	1e-5
Batch size	32
Image size	224x224
Validation split	30%
Gamma (FocalLoss)	2.0
Alpha (FocalLoss)	0.25
Layer dibekukan	0-120 dari ~155 total layer
Parameter MobileNetV2	~3.4 juta
Dataset	BUSI: 891 benign, 421 malignant, 266 normal

Alur Satu Kalimat untuk Pembuka Presentasi

"MediScan AI adalah sistem klasifikasi kanker payudara dari citra USG yang menggunakan transfer learning MobileNetV2 dengan dua fase training — frozen lalu fine-tuning — serta SparseFocalLoss untuk menangani dataset tidak seimbang, mencapai akurasi validasi 81.11% tanpa mengorbankan sensitivitas kelas malignant."

Selamat ujian! Fokus pada alur berpikir, bukan hafalan kode mentah. Jelaskan **mengapa** anda memilih setiap komponen, bukan hanya **apa** yang anda pilih.